Exemple de refactorisation backend

Structure proposée

backend/
app.js
server.js
routes/
auth.route.js
controllers/
auth.controller.js
services/
auth.service.js
repositories/
user.repository.js
middlewares/
asyncHandler.js
validateRequest.js
errorHandler.js
helpers/
jwt.js
config/
db.js
env.js

server.js

```
const app = require('./app');
const db = require('./config/db');
const PORT = process.env.PORT || 5000;

async function start() {
  try {
    await db.query('SELECT 1');
  } catch (err) {
    console.error('DB startup error:', err.message || err);
    process.exit(1);
  }

  app.listen(PORT, () => {
    console.log(`Server listening on port ${PORT}`);
  });
}

start();
```

app.js

```
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');
```

```
const authRouter = require('./routes/auth.route');
const errorHandler = require('./middlewares/errorHandler');

const app = express();

app.use(cors());
app.use(express.json());
app.use(helmet());
app.use(rateLimit({ windowMs: 15 * 60 * 1000, max: 100 }));

app.use('/api/auth', authRouter);

app.use(errorHandler);

module.exports = app;
```

auth.route.js

```
const router = require('express').Router();
const authController = require('../controllers/auth.controller');
const validateRequest = require('../middlewares/validateRequest');
const { registerSchema, loginSchema } = require('../validators/auth.validator');

router.post('/register', validateRequest(registerSchema), authController.register);
router.post('/login', validateRequest(loginSchema), authController.login);

module.exports = router;
```

auth.controller.js

```
const authService = require('../services/auth.service');

module.exports.register = async (req, res, next) => {
  try {
    const user = await authService.register(req.body);
    return res.status(201).json({ msg: 'Registered successfully', user });
  } catch (err) {
    next(err);
  }
};

module.exports.login = async (req, res, next) => {
  try {
    const token = await authService.login(req.body);
    return res.status(200).json({ msg: 'Login successfully', token });
  } catch (err) {
    next(err);
  }
};
```

auth.service.js

```
const userRepository = require('../repositories/user.repository');
const bcrypt = require('bcrypt');
const { generateToken } = require('../helpers/jwt');
const ApiError = require('../errors/ApiError');
```

```
module.exports.register = async ({ email, fullName, password }) => {
  const existing = await userRepository.findByEmail(email);
  if (existing) {
    throw ApiError.conflict('Email is already registered');
  }

  const hashedPassword = await bcrypt.hash(password, 10);
  const user = await userRepository.create({ email, fullName, password: hashedPassword });

  return {
    id: user.id,
    email: user.email,
    fullName: user.fullName,
    token: generateToken(user.id),
  };
};

module.exports.login = async ({ email, password }) => {
  const user = await userRepository.findByEmail(email);
  if (!user) {
    throw ApiError.badRequest('Invalid credentials');
  }

  const isValid = await bcrypt.compare(password, user.password);
  if (!isValid) {
    throw ApiError.unauthorized('Invalid credentials');
  }

  return {
    token: generateToken(user.id),
    user: { id: user.id, email: user.email },
  };
};
```

user.repository.js

```
const { query } = require('../config/db');

module.exports.findByEmail = async (email) => {
  const result = await query('SELECT id, email, fullName, password FROM users WHERE email = $1', [email]);
  return result.rows[0];
};

module.exports.create = async ({ email, fullName, password }) => {
  const result = await query(
    'INSERT INTO users (email, fullName, password) VALUES ($1, $2, $3) RETURNING id, email, fullName',
    [email, fullName, password]
  );
  return result.rows[0];
};
```

jwt.js

```
const jwt = require('jsonwebtoken');

module.exports.generateToken = (id) => {
  const secret = process.env.JWT_SECRET || process.env.SECRET_TOKEN;
  if (!secret) throw new Error('JWT secret missing');
```

```
  return jwt.sign({ id }, secret, { expiresIn: '30d', algorithm: 'HS256' });
};
```